

Self-Discovery Questions

Research and find answers to these common interview questions and real-world architectural considerations based on the Build a Website on Google Cloud project.

Cloud Run (Serverless)

1. Cloud Run can automatically "scale to zero" when there is no traffic. What is a "**cold start**," and how does it affect the user experience?

A "cold start" in Cloud Run is the delay that occurs when a new container instance must be initialized from scratch to handle a request, typically after the service has scaled to zero due to no traffic

Effect on user experience:

Increased latency

User-visible delays

Impact on critical workflows

2. By default, Cloud Run assigns a random `run.app` URL. How do you map a custom domain name (like `www.my-company.com`) to a Cloud Run service?

To map a custom domain to a Cloud Run service, verify ownership of the domain in Google Cloud, then use the Cloud Run "Custom Domains" feature. Cloud Run provides DNS records that must be added to your domain registrar. Once DNS propagation is complete, traffic to the custom domain, such as `www.my-company.com`, is securely routed to the Cloud Run service with managed SSL certificates automatically provisioned by Google.

3. What is the difference between setting Cloud Run concurrency to 1 versus setting it to 80 ? How does this impact billing?

Concurrency defines how many requests a single Cloud Run instance can process simultaneously. With concurrency set to 1, each instance handles one request at a time, improving isolation but potentially creating more instances. With concurrency set to 80, fewer instances are needed because each can handle multiple requests. Higher concurrency can reduce infrastructure costs, while lower concurrency may improve response times for CPU-intensive workloads.

Compute Engine (Virtual Machines)

4. Google Cloud offers both **Managed** Instance Groups (MIGs) and **Unmanaged** Instance Groups. What is the difference, and when would you actually use an Unmanaged group?

Unmanaged instances are a group of heterogeneous instances which user has fine grained control over. Each instance can be configured individually by user. They are used when instances have different configurations, legacy workloads, or specialized applications that require individual management and cannot easily fit into a standardized template. Managed Instance Groups (MIGs) use an instance template to automatically create, scale, update, and repair identical VM instances. They support features such as autoscaling, load balancing, and auto-healing.

5. We used a Global HTTP(S) Load Balancer in the lab. How does this differ from a Regional Network (TCP/UDP) Load Balancer?

A Global HTTP(S) Load Balancer operates at Layer 7 and distributes web traffic globally based on URL paths, hostnames, and other application-level rules. It provides SSL termination and content-aware routing. A Regional Network Load Balancer operates at Layer 4 and balances TCP or UDP traffic within a specific region. It is better suited for non-HTTP applications requiring low-latency network-level load balancing.

6. If a VM in a Managed Instance Group fails a health check, exactly what steps does the MIG take to auto-heal it? Does it reboot the machine or delete it?

When a VM in a Managed Instance Group fails its configured health check, the MIG identifies it as unhealthy and automatically repairs it. Instead of simply rebooting the VM, the default action is to delete the unhealthy instance and recreate a new one from the instance template. This ensures the replacement VM matches the desired configuration and helps maintain application availability without manual intervention.

Google Kubernetes Engine (Containers)

7. In Kubernetes, what is the exact difference between a `ClusterIP` , a `NodePort` , and a `LoadBalancer` service?

`ClusterIP` exposes a service only within the Kubernetes cluster and is the default service type. `NodePort` exposes the service on a static port across all cluster nodes, allowing external access through the node's IP address. `LoadBalancer` creates an external cloud load balancer that automatically routes internet traffic to the service, providing the easiest and most scalable public access option.

8. What is a Kubernetes "**Namespace**," and why would a large enterprise with multiple development teams use them?

A Namespace is a logical partition within a Kubernetes cluster that separates resources and workloads. Large enterprises use namespaces to isolate development, testing, and production environments or different teams sharing the same cluster. Namespaces improve organization, security, and resource management by enabling role-based access control and resource quotas, preventing teams from interfering with each other's applications.

9. We scaled our GKE deployments manually using `kubectl scale`. How does a **Horizontal Pod Autoscaler (HPA)** automate this, and what metrics does it use to make decisions?

The Horizontal Pod Autoscaler automatically adjusts the number of running pods based on demand. It continuously monitors metrics such as CPU utilization, memory consumption, or custom application metrics. When usage exceeds a defined threshold, HPA increases the number of pods; when demand decreases, it scales them down. This helps applications maintain performance while optimizing resource usage and infrastructure costs.

10. What happens to the files or data stored inside a Docker container when a GKE Pod crashes and is recreated? Where should you store data if you want it to survive a crash?

Data stored inside a container's filesystem is ephemeral and is lost when a pod crashes or is recreated. Since Kubernetes may replace pods at any time, important data should not be stored within containers. To ensure persistence, applications should use Persistent Volumes, Persistent Volume Claims, cloud storage services, or managed databases so data remains available even after restarts or failures.

Architecture, CI/CD, & Migrations

11. We used Cloud Build to create images and Artifact Registry to store them. What is the difference between a CI/CD pipeline (like Cloud Build) and a Container Registry?

A CI/CD pipeline, such as Cloud Build, automates software building, testing, and deployment processes. It transforms source code into deployable artifacts and ensures quality through automated workflows. A Container Registry, such as Artifact Registry, is a storage repository for container images and artifacts. The CI/CD pipeline creates the images, while the registry securely stores and distributes them for deployment.

12. In a microservices architecture, how do you handle database transactions that span across multiple services (e.g., an order is placed, but inventory needs to be updated)?

In microservices architectures, distributed transactions are typically handled using patterns like Saga rather than traditional database transactions. Each service performs its own transaction and publishes events for other services to act upon. If a step fails, compensating actions are executed to reverse previous operations. This approach improves scalability and resilience while maintaining eventual consistency across independently managed services.

13. What is an **"API Gateway,"** and why is it highly recommended to put one in front of your microservices?

An API Gateway acts as a centralized entry point for client requests before they reach backend microservices. It provides features such as authentication, authorization, rate limiting, request routing, monitoring, and SSL termination. Using an API Gateway reduces complexity for clients, improves security, and enables consistent policy enforcement, making it easier to manage large-scale microservice ecosystems.

14. How do you securely pass sensitive information, like database passwords or API keys, to a Kubernetes Pod without hardcoding them in your `.env` file or Docker image?

Sensitive information such as database passwords and API keys should be stored in Kubernetes Secrets rather than hardcoded in code, environment files, or container images. Secrets can be mounted as environment variables or files within pods. For enhanced security, organizations often integrate Kubernetes with secret management systems like Google Secret Manager to securely store, rotate, and audit access to credentials.

15. What is the "**Sidecar**" container pattern in Kubernetes, and what is a common real-world use case for it?

The Sidecar pattern involves running an additional container alongside the main application container within the same Kubernetes pod. The sidecar provides supporting functionality without modifying the application itself. Common use cases include log collection, monitoring, security proxies, service mesh components, and configuration synchronization. Since both containers share networking and storage resources, they can work together closely and efficiently.
